STYLUS

Retail de moda con compras inteligentes asistidas por IA en tiempo real

Plataforma Big Data en Google Cloud sobre el dataset público H&M (Kaggle): ingesta batch y streaming, lakehouse medallion, analítica, recomendaciones y asistente con Vertex AI Gemini

ENTREGA 1

Examen (E): Caso de estudio y sustentación técnica, económica y de resiliencia
Proyecto Parte 1 (P1): Blueprint, diseño de ingesta y almacenamiento

Integrante	Perfil en el proyecto
Estuardo Oliver	ML / Analytics: asistente conversacional, analítica y recomendaciones
Winston Quispe	Cloud Computing: infraestructura GCP, ingesta y streaming
Nicole Arenas	Datos + API + Modelado: capas medallion, modelo dimensional y API
Gianmarco Mejia	Dashboard / BI: KPIs, tableros y visualización

Fecha de entrega: 24 de octubre de 2026 · Versión 1.1
Repositorio: github.com/estuardoo/STYLUS · Proyecto GCP: tranquil-leaf-512618-b9

Control del documento

Campo	Detalle
Documento	Entrega 1: Examen (E) + Proyecto Parte 1 (P1)
Versión	1.1 (24-oct-2026): nombre STYLUS, configuración en JSON + panel web, repositorio y proyecto GCP reales
Fuente de requisitos	UTEC, “Entregables del Proyecto Final: Taller de Arquitectura Big Data” (PDF, 3 páginas)
Repositorio	Estructura y código de soporte en el repositorio del proyecto (ver Anexo A)
Datos	H&M Personalized Fashion Recommendations (Kaggle). Única fuente de datos del proyecto
Precios	Lista pública de Google Cloud, región us-central1, consultados el 22-oct-2026

Convención de estado. En este documento distinguimos tres niveles de madurez para no sobredeclarar avances: **diseñado** (decisión documentada), **implementado** (código o configuración en el repositorio, probado localmente) y **desplegado** (funcionando en GCP con evidencia). Para la Entrega 1 el curso exige diseño; lo implementado se indica explícitamente.

Índice

1. Resumen ejecutivo....4
2. Contexto de negocio y alcance....4
2.1 Problema....4
2.2 Objetivos....4
2.3 Alcance y fuera de alcance....5
2.4 Supuestos....5
3. Equipo y roles....5
4. Inventario y perfil de datos....6
4.1 Hallazgos de calidad....6
4.2 Patrones de negocio útiles para el diseño....6
4.3 Decisión sobre la fuente Amazon Reviews 2023....6
5. Stack tecnológico y justificación....7
6. Arquitectura end-to-end....8
6.1 Flujo batch....8
6.2 Flujo streaming....9
6.3 Integración multifuente....9
7. Definición de ingesta y almacenamiento....9
7.1 Mecanismos de captura....9
7.2 Motores de almacenamiento y modelo medallion....9
7.3 Implementado para esta entrega....10
8. Escalabilidad y performance....10
8.1 Degradación controlada....11
8.2 Metas de performance....11
9. Justificación económica y optimización de costos....11
9.1 Escenarios....11
9.2 Palancas de optimización aplicadas....12
9.3 Automatización de costos....12
10. Plan de recuperación y alta disponibilidad (DRP/HA)....12
10.1 Prototipo frente a producción....13
11. KPIs de éxito y performance....13
12. Seguridad y gobierno....14
13. Plan de trabajo por entregas....14
14. Riesgos y mitigaciones....14
Anexo A. Repositorio y reproducibilidad....14
Referencias....15

1. Resumen ejecutivo

Proponemos una plataforma analítica integral para un retailer de moda, construida sobre Google Cloud Platform (GCP) y alimentada con el dataset público de H&M: 31.8 millones de líneas de venta de dos años, 1.37 millones de clientes y 105,542 artículos con sus imágenes. La plataforma convierte ese histórico en tres productos de valor: analítica de ventas para gerencia, recomendaciones personalizadas y un asistente de compras conversacional que propone productos reales del catálogo.

La arquitectura sigue el patrón lakehouse medallion. Cloud Storage guarda los datos originales intactos (Bronze) y sus versiones limpias en Parquet (Silver/Gold); BigQuery actúa como data warehouse; Pub/Sub y Dataflow procesan eventos en tiempo real; Cloud Run expone la API y la web; Cloud SQL mantiene el estado transaccional del carrito y los pedidos; y Vertex AI Gemini 3.1 Flash-Lite impulsa el asistente. Todo se declara con Terraform y se despliega desde GitHub.

> **Resultados clave del diseño**
> Costo estimado de producción a escala H&M: USD 370.83 al mes, equivalente a USD 0.00098 por pedido procesado. Año 1 (11 meses normales + 1 mes pico Black Friday): USD 5,482. Costo de desarrollo y demo del curso: USD 19.77. Todos los componentes de cómputo son serverless o autoescalables, por lo que el costo crece con la demanda y no con la capacidad reservada.

Los tres pilares que pide el examen se resuelven así:

- **Escalabilidad:** desacople por Pub/Sub, autoescalado de Dataflow y Cloud Run, y tablas particionadas en BigQuery; modelamos un pico de 5 veces el tráfico normal.
- **Economía:** calculadora de costos automatizada (supuestos en YAML, Excel con fórmulas, refresco de precios desde la API oficial de facturación), presupuestos con alertas y medición del costo real por pedido.
- **Resiliencia:** Bronze inmutable y versionado como punto de reconstrucción, retención de mensajes en Pub/Sub, backups y PITR en Cloud SQL, con RPO/RTO definidos por componente.

2. Contexto de negocio y alcance

2.1 Problema

Un retailer de moda con canal físico y online acumula millones de transacciones, pero las usa casi solo para reportes contables. Las preguntas de negocio que importan quedan sin respuesta oportuna: qué recomendar a cada cliente, qué productos empujar en temporada alta, cuánto cuesta realmente procesar un pedido y cómo reaccionar en minutos, no en días, ante un pico como Black Friday.

2.2 Objetivos

- Integrar el histórico de ventas (batch) con eventos de navegación y compra en tiempo real (streaming) en una única plataforma gobernada.
- Entregar analítica confiable mediante un modelo dimensional en BigQuery y tableros en Looker Studio.
- Ofrecer recomendaciones personalizadas y un asistente conversacional que solo propone artículos reales del catálogo.

- Mantener el costo por pedido medible, bajo y predecible, con alertas automáticas de presupuesto.

2.3 Alcance y fuera de alcance

En alcance	Fuera de alcance
Dataset H&M completo: artículos, clientes, transacciones e imágenes	Otros datasets (Amazon Reviews se descartó; ver §4.3)
Ingesta batch a GCS y BigQuery; streaming vía Pub/Sub y Dataflow	Integración con sistemas reales de una tienda (POS, ERP)
Capas Bronze, Silver y Gold; modelo estrella	Pagos reales: el saldo es simulado y no usa tarjetas ni bancos
API, recomendador, asistente Gemini y web demo	Alta disponibilidad multirregional desplegada (se diseña; el prototipo es de una región)
Tableros gerenciales y operativos; calculadora de costos	Recomendador visual con imágenes (queda como extensión)

2.4 Supuestos

- La compra es una simulación: cada recarga de saldo es explícita y auditable; no hay dinero real.
- Los eventos de 2026 se generan re-emitiendo transacciones reales de H&M con un nuevo event_time; la fecha original se conserva en source_t_dat y todo evento lleva is_synthetic = true.
- El precio de H&M viene normalizado (máximo 0.59); la web aplica un factor de precio demo configurable, señalado como sintético.
- Región principal us-central1 (precios de nivel 1 y disponibilidad de Gemini); los datos son públicos y anonimizados, por lo que no hay restricción de residencia.

3. Equipo y roles

Los roles se asignaron por perfil profesional. Cada tarea tiene un responsable único y un apoyo; el seguimiento detallado está en la matriz de auditoría (docs/gestion/Matriz_Auditoria_Proyecto.xlsx).

Integrante	Perfil	Responsabilidades principales
Estuardo Oliver	ML / Analytics	Asistente con Vertex AI Gemini (function calling sobre catálogo), recomendador (baseline y contenido), features analíticas, evaluación de modelos, guion de demo y documento técnico
Winston Quispe	Cloud Computing	Terraform, proyecto GCP e IAM, buckets y Pub/Sub, pipeline Dataflow, CI/CD con GitHub Actions, monitoreo, DRP/HA y automatización de costos
Nicole Arenas	Datos + API + Modelado	Ingesta batch y Bronze, limpieza Silver y calidad, modelo estrella Gold, diccionario de datos, API FastAPI y base transaccional Cloud SQL
Gianmarco Mejia	Dashboard / BI	Catálogo de KPIs, calculadora y tablero de costos, diagramas, tableros Looker Studio (gerencial y operativo) y presentación final

4. Inventario y perfil de datos

Antes de diseñar, perfilamos los archivos reales en data/raw. Las cifras de esta sección provienen de una lectura completa de cada archivo, no de la documentación del dataset.

Archivo	Tamaño	Filas	Contenido y clave
transactions_train.csv	3.49 GB	31,788,324	Venta por línea: t_dat, customer_id, article_id, price, sales_channel_id. 20-sep-2018 a 22-sep-2020 (734 días)
customers.csv	207 MB	1,371,980	customer_id (hash SHA-256), edad, estado de membresía, código postal hasheado
articles.csv	36 MB	105,542	article_id, 47,224 product_code, 131 tipos de producto, 5 grupos de índice, descripción
images/	29 GB	105,100 jpg	Una imagen por artículo en carpetas por prefijo (442 artículos sin imagen)
sample_submission.csv	270 MB	1,371,980	Formato de predicción de la competencia; no se usa como dato de negocio

Tabla 1. Inventario de la fuente H&M (medido el 22-oct-2026).

4.1 Hallazgos de calidad

Hallazgo	Medición	Tratamiento en Silver
Edad faltante	15,861 clientes (1.2%)	Imputar como desconocida; segmento 'sin edad'
FN y Active vacíos	895,050 y 907,576 nulos	Interpretar vacío como 0 (bandera no activada)
Estado de membresía y newsletter	6,062 y 16,009 nulos	Categoría 'NONE'
Descripción de artículo	416 nulos	Usar nombre y tipo de producto como respaldo
Precio normalizado	Máximo 0.59; sin moneda	Se conserva; factor de precio demo solo en la web
Pedido no explícito	Sin order_id	Pedido = cliente + día: 9,080,179 pedidos, 3.5 líneas de media

Tabla 2. Calidad de datos y regla de tratamiento.

4.2 Patrones de negocio útiles para el diseño

- **Volumen:** ~1.32 millones de líneas y ~377 mil pedidos por mes; el mes más alto (junio de 2019, 1.91 millones de líneas) es 1.44 veces el promedio.
- **Canal:** 70.4% de las líneas son del canal online (22.4 millones) y 29.6% de tienda física.
- **Caso bebé:** 34,711 artículos pertenecen al grupo Baby/Children (8,875 de ellos con tallas de bebé), por lo que el caso de demo “necesito algo para mi bebé” se resuelve con catálogo real.
- **Privacidad:** customer_id y postal_code ya vienen seudonimizados con hash; no hay nombres, correos ni teléfonos.

4.3 Decisión sobre la fuente Amazon Reviews 2023

Evaluamos Amazon Reviews 2023 como segunda fuente. La descartamos porque sus identificadores (ASIN, user_id) no comparten espacio con los de H&M y tres de las seis categorías descargadas no tienen relación con moda. Unir ambas fuentes sin una clave común habría requerido inventar correspondencias. Los archivos quedan archivados fuera del repositorio.

5. Stack tecnológico y justificación

Priorizamos componentes de GCP vistos en clase, serverless y con pago por uso. Cada decisión se tomó frente a una alternativa concreta.

Capa	Servicio elegido	Por qué	Alternativa descartada
Data Lake	Cloud Storage (2 buckets privados)	Barato (USD 0.020/GB-mes), durable, versionado; desacopla almacenamiento de cómputo	HDFS en Dataproc: paga nodos encendidos
Data Warehouse	BigQuery	Serverless, SQL estándar, particionado y clustering; 1 TiB de consulta gratis al mes	Cloud SQL para analítica: no escala a 31.8M filas con consultas ad hoc
Ingesta streaming	Pub/Sub + suscripción BigQuery	Absorbe picos, retiene mensajes, DLQ nativa; la suscripción BQ evita un job permanente	Kafka autogestionado: operación y costo fijo
Procesamiento streaming	Dataflow (Apache Beam)	Ventanas, deduplicación, datos tardíos y autoescalado; mismo código batch/stream	Cloud Functions: sin estado ni ventanas
Procesamiento batch	Serverless for Apache Spark (Dataproc) + BigQuery SQL	Spark sin clúster para limpieza a Parquet; SQL para Gold	Clúster Dataproc permanente
OLTP	Cloud SQL PostgreSQL	ACID para carrito, pedidos y ledger del saldo	BigQuery: no es transaccional
API y web	Cloud Run	Contenedores que escalan a cero; 2M requests gratis/mes	GKE: sobredimensionado para el caso
IA generativa	Vertex AI Gemini 3.1 Flash-Lite	Gemini más económico vigente en Vertex AI (USD 0.25/1.50 por millón de tokens); salida estructurada	LLM propio en GPU: costo fijo alto
BI	Looker Studio	Sin licencia, conector nativo a BigQuery	Tableau/Power BI: licencias
IaC y CI/CD	Terraform + GitHub Actions (WIF)	Infraestructura reproducible; sin llaves permanentes	Configuración manual en consola

Tabla 3. Decisiones tecnológicas (ADR resumidos).

> **Sobre el costo de Vertex AI Gemini**
>
> Gemini en Vertex AI no es gratuito, pero su costo es marginal: una conversación típica (≈3,000 tokens de entrada y 400 de salida) cuesta alrededor de USD 0.0014. Mil conversaciones de prueba cuestan unos USD 1.40, y los créditos de bienvenida de GCP (USD 300 para cuentas nuevas) cubren holgadamente el proyecto. Se limita además con cuotas por minuto y con el presupuesto con alertas descrito en §9.3.

6. Arquitectura end-to-end

La Figura 1 muestra las cinco capas que exige P1 (ingesta, almacenamiento, cómputo y exposición, más las fuentes) y los componentes transversales de seguridad, operación y costos.

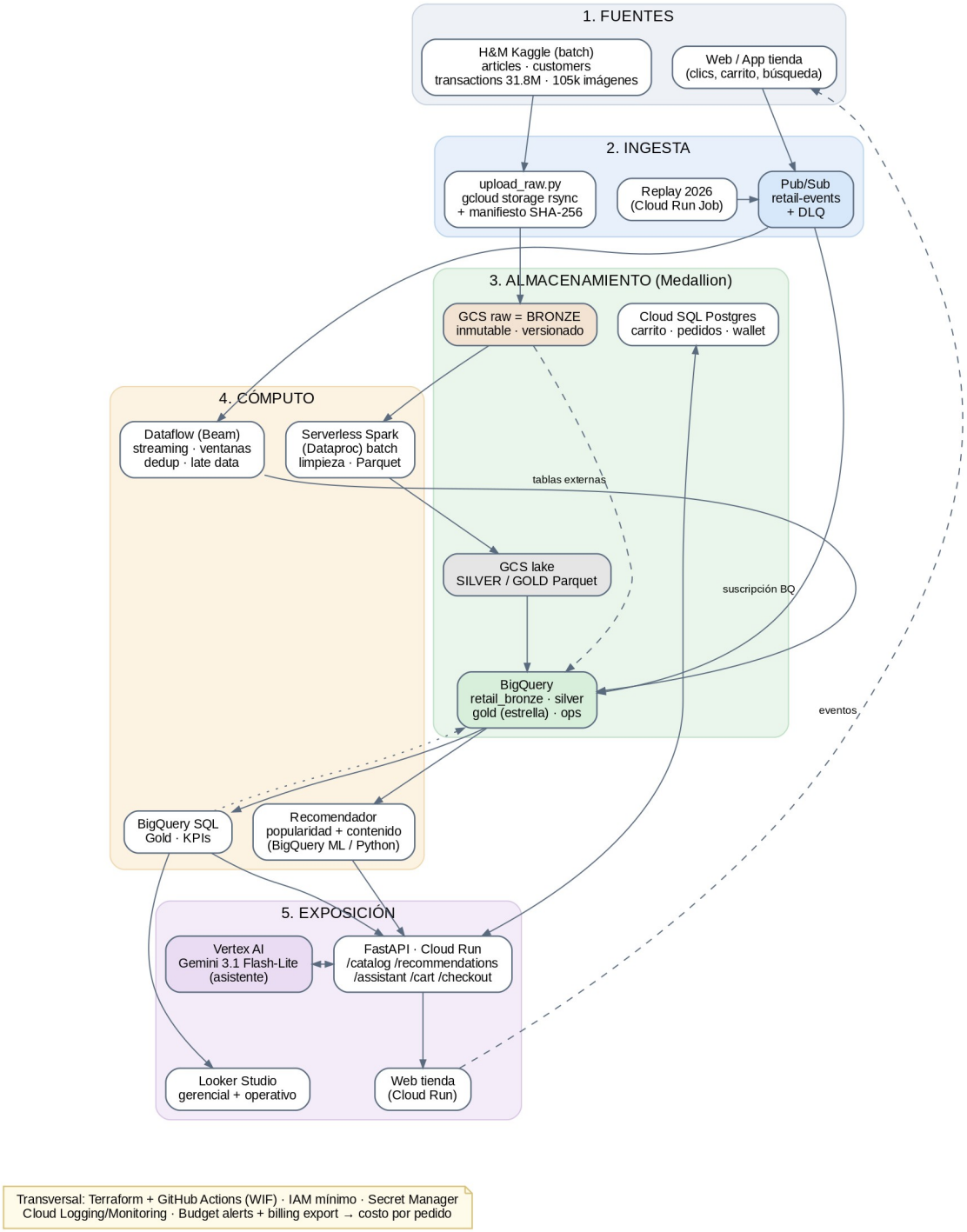


Figura 1. Arquitectura end-to-end (fuente regenerable: docs/architecture/arquitectura_vertical.dot).

6.1 Flujo batch

- upload_raw.py calcula un manifiesto (SHA-256, bytes, filas, columnas) y sincroniza data/raw con el bucket raw mediante gcloud storage rsync. Reejecutar no duplica datos.

- BigQuery lee los CSV de Bronze como tablas externas y materializa transactions particionada por día y agrupada por cliente y artículo (sql/bronze/01_bronze_hm.sql).
- Serverless Spark limpia y tipa hacia Silver en Parquet; BigQuery SQL construye el modelo estrella Gold.

6.2 Flujo streaming

- El publicador replay_publisher.py re-emite transacciones reales como eventos de 2026 hacia el tópico retail-events. Cada evento lleva event_id determinista (idempotencia), event_time 2026, source_t_dat original y la marca is_synthetic.
- Ruta económica: una suscripción Pub/Sub → BigQuery escribe los mensajes crudos en retail_bronze.events_raw sin ningún job intermedio.
- Ruta enriquecida (ventana de demo): Dataflow lee la suscripción dedicada, deduplica por event_id, agrega en ventanas de 1 minuto, maneja datos tardíos y envía errores a la DLQ.

6.3 Integración multifuente

El histórico batch y el flujo streaming se cruzan por las claves naturales de H&M (article_id y customer_id), que son las mismas en ambos caminos porque el streaming re-emite registros del propio dataset. Para no mezclar historia con simulación separamos dos relojes: source_t_dat (fecha real, nunca se reescribe) y event_time (fecha simulada 2026). Toda tabla lleva _source_system y _ingested_at para trazabilidad.

7. Definición de ingesta y almacenamiento

7.1 Mecanismos de captura

Flujo	Mecanismo	Frecuencia	Garantías
Histórico H&M (CSV + imágenes)	gcloud storage rsync + manifiesto SHA-256	Carga inicial y reprocesos	Idempotente; checksum y conteo por archivo
Eventos de compra/navegación	Pub/Sub (tópico retail-events)	Continuo, segundos	Entrega al menos una vez; dedup por event_id; DLQ tras 5 intentos
Pedidos y saldo de la web	Cloud SQL (escritura transaccional)	En línea	ACID; clave de idempotencia por checkout
Costos de la plataforma	Billing export a BigQuery	Diario	Fuente oficial de facturación

Tabla 4. Mecanismos de captura.

7.2 Motores de almacenamiento y modelo medallion

Capa	Motor y ubicación	Formato	Contenido	Política
Bronze	GCS raw (gs://…-retail-raw/hm/) + BigQuery retail_bronze	CSV/JPG originales; tablas externas y events_raw	Copia exacta de H&M y eventos crudos	Inmutable, versionado, imágenes a Nearline a los 30 días
Silver	GCS lake + BigQuery retail_silver	Parquet particionado por fecha	Datos tipados, sin duplicados, nulos tratados	Reproducible desde Bronze

Capa	Motor y ubicación	Formato	Contenido	Política
Gold	BigQuery retail_gold	Tablas nativas particionadas	Modelo estrella: fact_sales, fact_orders, dim_article, dim_customer, dim_date, dim_channel; features	Optimizado para consumo
Operación	BigQuery retail_ops	Tablas y vistas	Calidad, lag de streaming, costos reales	Retención 13 meses
Transaccional	Cloud SQL PostgreSQL	Relacional	Usuarios demo, carrito, pedidos, wallet y ledger	Backups diarios + PITR

Tabla 5. Motores de almacenamiento por capa.

No usamos una base NoSQL (Bigtable o Firestore): el acceso por clave de baja latencia que justificaría una se cubre con Cloud SQL para el estado del usuario y con BigQuery más una caché en memoria de la API para las recomendaciones precalculadas. Una NoSQL agregaría costo fijo sin un caso de uso que la requiera.

7.3 Implementado para esta entrega

Artefacto	Ruta en el repositorio	Estado
Infraestructura: buckets privados, datasets por capa, Pub/Sub + DLQ + suscripción BQ, cuentas de servicio, presupuesto	infra/terraform/	Implementado; pendiente de plan/apply en GCP
Ingesta batch con manifiesto	ingestion/batch/upload_raw.py	Implementado y probado localmente
Bronze en BigQuery	sql/bronze/01_bronze_hm.sql	Implementado; pendiente de ejecución en GCP
Publicador streaming 2026	ingestion/streaming/ replay_publisher.py	Implementado; pruebas unitarias OK
Calculadora de costos automatizada	cost/	Implementado y ejecutado

8. Escalabilidad y performance

Dimensionamos con los datos reales. En operación normal H&M procesa ~12,400 pedidos diarios (~43,000 líneas). Suponiendo 20 eventos de navegación por línea comprada, el tráfico medio es de ~10 eventos por segundo. Para Black Friday modelamos 5 veces el volumen mensual y una hora punta 50 veces superior a la media: ~500 eventos por segundo.

Componente	Cómo escala	Comportamiento en el pico	Límite / control
Pub/Sub	Automático, sin aprovisionar	Absorbe ráfagas y retiene 7 días si el consumidor se atrasa	Cuota de publicación por región; alerta por backlog
Dataflow	Autoescalado horizontal de workers	Pasa de 1 a N workers según backlog; Streaming Engine separa estado y cómputo	max_num_workers acotado para el presupuesto
BigQuery	Slots bajo demanda	Consultas sobre particiones del día; clustering por cliente y artículo reduce bytes escaneados	Costo por TiB; require_partition_filter evita escaneos completos
Cloud Run (API/web)	Instancias por concurrencia	De 0 a decenas de instancias en segundos; 80 requests concurrentes por instancia	max-instances y min-instances=1 en temporada alta

Componente	Cómo escala	Comportamiento en el pico	Límite / control
Cloud SQL	Vertical + réplicas de lectura	Solo escrituras de carrito y pedidos; las lecturas analíticas van a BigQuery	Pool de conexiones; réplica para lecturas
Vertex AI Gemini	Servicio gestionado	Respuestas en ~1 s; caché de recomendaciones por segmento	Cuota por minuto; respaldo a recomendador por reglas

Tabla 6. Estrategia de escalado por componente.

8.1 Degradación controlada

- Si Gemini supera su cuota, el asistente responde con recomendaciones por popularidad del segmento, sin bloquear la compra.
- Si Dataflow se atrasa, la suscripción directa a BigQuery sigue registrando los eventos crudos; los agregados se recalculan después.
- Si Cloud SQL se satura, el checkout responde con reintento y clave de idempotencia; nunca cobra dos veces.

8.2 Metas de performance

Latencia p95 de la API menor a 500 ms (sin asistente) y menor a 3 s con Gemini; lag de streaming p95 menor a 60 s entre publicación y disponibilidad en BigQuery; consultas Gold de tablero menores a 5 s.

9. Justificación económica y optimización de costos

9.1 Escenarios

La calculadora (cost/assumptions.yaml → cost/output/cost_estimate.xlsx) estima el costo mensual por servicio en cuatro escenarios. Los volúmenes de producción salen del perfil real de H&M: 377,000 pedidos al mes. Precios de lista en us-central1, consultados el 22-oct-2026, aplicando los niveles gratuitos mensuales.

Servicio	Desarrollo (curso)	Demo / sustentación	Producción (escala H&M)	Pico Black Friday (5×)
Cloud Storage	USD 0.80	USD 0.54	USD 1.50	USD 1.70
BigQuery	USD 0.04	USD 0.10	USD 44.75	USD 120.15
Pub/Sub	USD 0.05	USD 0.10	USD 2.31	USD 13.11
Dataflow	USD 0.00	USD 3.95	USD 120.21	USD 360.64
Serverless Spark	USD 0.96	USD 0.48	USD 3.60	USD 7.20
Cloud Run	USD 0.00	USD 0.00	USD 34.26	USD 195.36
Cloud SQL	USD 4.70	USD 3.50	USD 28.95	USD 29.80
Vertex AI Gemini	USD 1.50	USD 2.75	USD 135.00	USD 675.00
Artifact Registry	USD 0.15	USD 0.15	USD 0.25	USD 0.25
Total mensual	USD 8.20	USD 11.57	USD 370.83	USD 1,403.21
Pedidos / mes	5,000	20,000	377,000	1,885,000
Costo por pedido	USD 0.00164	USD 0.00058	USD 0.00098	USD 0.00074

Tabla 7. Costo mensual estimado por servicio y escenario (USD).

> **Costo por pedido**
> En producción la plataforma cuesta USD 0.00098 por pedido (menos de una décima de centavo). En el pico de Black Friday baja a USD 0.00074, porque los costos fijos (Cloud SQL, almacenamiento) se reparten entre más pedidos y el resto escala con el uso. El año 1 completo se estima en USD 5,482.

9.2 Palancas de optimización aplicadas

- **Serverless y escala a cero:** Cloud Run, Serverless Spark y BigQuery no cobran sin uso.
- **Ruta streaming sin job permanente:** la suscripción Pub/Sub → BigQuery cuesta USD 50/TiB sin cargos de ingesta; Dataflow se enciende solo cuando se necesita enriquecimiento con ventanas.
- **Menos bytes escaneados:** particionado por fecha, clustering y filtro de partición obligatorio; Parquet columnar en Silver.
- **Ciclo de vida del almacenamiento:** las 105,100 imágenes (29 GB) pasan a Nearline a los 30 días, a la mitad del precio.
- **Modelo de IA más barato:** Gemini 3.1 Flash-Lite (el modelo se cambia desde config/control_plataforma.json) y caché de respuestas frecuentes.
- **Región de nivel 1:** us-central1 tiene las tarifas más bajas para Cloud Run y Dataflow.

9.3 Automatización de costos

El control de costos no depende de revisiones manuales. Implementamos tres mecanismos que se refuerzan entre sí:

Mecanismo	Qué hace	Dónde
1. Estimador automatizado	Supuestos en un único YAML; un script genera el Excel con fórmulas y el resumen. Con --refresh-prices consulta la API Cloud Billing Catalog y actualiza las tarifas con fecha. Corre en cada push (CI).	cost/estimate_costs.py (--refresh-prices)
2. Presupuesto con alertas	Budget por proyecto con umbrales del 50%, 90% y 100% (real) y 100% (pronosticado); publica en un tópico Pub/Sub para notificar o apagar recursos de demo.	infra/terraform/main.tf (google_billing_budget)
3. Costo real por pedido	Billing export a BigQuery + vista que cruza costo diario por servicio con pedidos procesados en Gold; alimenta el tablero de costos.	cost/real_cost_per_order.sql

Tabla 8. Mecanismos de automatización de costos.

El ciclo se cierra comparando lo estimado (mecanismo 1) con lo facturado (mecanismo 3) y recalibrando los supuestos; esa comparación forma parte de la Entrega 3.

Nota: *son estimaciones con precios de lista, no una cotización. Deben validarse en la calculadora oficial https://cloud.google.com/products/calculator; el costo final depende del uso real.*

10. Plan de recuperación y alta disponibilidad (DRP/HA)

El principio central es que todo dato derivado puede reconstruirse desde Bronze, que es inmutable y versionado. Por eso la pérdida de cualquier capa de cómputo o de Silver/Gold se resuelve reprocesando, y la protección se concentra en Bronze y en el estado transaccional.

Componente	Falla considerada	Protección	RPO	RTO
GCS raw (Bronze)	Borrado o sobrescritura accidental	Versionado de objetos, acceso uniforme, sin acceso público; durabilidad 11 nueves	0	< 15 min
BigQuery Silver/Gold	Tabla dañada o borrada	Time travel 7 días + reproceso desde Bronze	0	< 1 h
Pub/Sub	Consumidor caído	Retención de mensajes no confirmados (7 días), DLQ, seek para reprocesar	0	< 15 min
Dataflow	Falla del job o de workers	Reemplazo automático de workers; relanzamiento desde plantilla; la suscripción BQ sigue capturando	0	< 30 min
Cloud SQL	Corrupción o falla de zona	Backup diario + recuperación a un punto en el tiempo (PITR); en producción, instancia HA regional	≤ 5 min	< 1 h (prototipo) / ~1 min (HA)
Cloud Run	Falla de zona	Servicio regional multizona por defecto	N/A	Segundos
Región completa	Caída de us-central1	Terraform reproduce todo en una segunda región; Bronze en dual-region en producción	≤ 24 h (prototipo)	< 4 h
Estado de Terraform	Pérdida o conflicto	Bucket GCS versionado con bloqueo de estado	0	< 15 min

Tabla 9. Matriz de fallas, RPO y RTO.

10.1 Prototipo frente a producción

El prototipo del curso es de una sola región para mantener el costo bajo; no lo presentamos como alta disponibilidad probada. Para producción, el diseño agrega Cloud SQL en modo HA regional, Bronze en un bucket dual-region, BigQuery en la multirregión US y Cloud Run en dos regiones detrás de un balanceador global. La Entrega 3 incluirá una prueba real de restauración de Cloud SQL y de una versión de objeto en GCS, con el RTO medido.

11. KPIs de éxito y performance

Dimensión	KPI	Meta	Fuente
Datos	Filas cargadas vs manifiesto	100% conciliado	retail_ops + manifiesto
Datos	Filas rechazadas en Silver	< 0.5% y todas contabilizadas	Reporte de calidad
Streaming	Lag p95 publicación → BigQuery	< 60 s	events_raw.publish_time vs ingesta
Streaming	Mensajes en DLQ	< 0.1%	Monitoreo Pub/Sub
Modelo	MAP@12 del recomendador vs baseline de popularidad	Mejora > 10%	Evaluación con split temporal
API	Latencia p95 / disponibilidad	< 500 ms / ≥ 99.5% en ventana de prueba	Cloud Monitoring
Negocio (simulado)	Conversión, ticket medio, pedidos	Visibles por día y canal	Gold + Looker Studio
Costos	USD por pedido	≤ USD 0.0012 (estimado + 20%)	Billing export

Tabla 10. Catálogo de KPIs.

12. Seguridad y gobierno

- Buckets privados con acceso uniforme y prevención de acceso público forzada; cifrado en reposo y TLS en tránsito por defecto.
- Cuatro cuentas de servicio con mínimo privilegio (ingesta, pipelines, API y despliegue); GitHub Actions usa Workload Identity Federation, sin llaves JSON.
- Secretos en Secret Manager; .gitignore y una regla de CI impiden versionar datos, credenciales o estado de Terraform.
- Datos de clientes ya seudonimizados; los eventos sintéticos se identifican con is_synthetic y simulation_run_id.
- El asistente solo puede proponer artículos existentes y requiere confirmación explícita del usuario antes del checkout.

13. Plan de trabajo por entregas

Paso	Fecha	Entregable	Contenido principal
1	24-oct-2026	Examen + P1	Este documento: stack, escalabilidad, costos, DRP/HA, arquitectura, ingesta y almacenamiento; repositorio base
2	07-nov-2026	P2	Infraestructura desplegada, Bronze/Silver/Gold, replay streaming con Dataflow, modelo estrella, recomendador baseline
3	20/21-nov-2026	P3 + sustentación	API en Cloud Run, asistente Gemini, wallet en Cloud SQL, tableros, demo end-to-end, slides y costo real

Tabla 11. Hitos. El detalle por tarea y responsable está en la matriz de auditoría.

14. Riesgos y mitigaciones

Riesgo	Impacto	Mitigación
Gasto inesperado en nube	Alto	Presupuesto con alertas, Dataflow solo en demo, checklist de apagado
Subida lenta de 33 GB desde la red local	Medio	rsync reanudable; imágenes en segundo plano; CSV primero
Cuotas de Gemini o Dataflow en cuenta nueva	Medio	Solicitar cuota temprano; respaldo por reglas
Plazo corto entre entregas	Alto	MVP por hito; responsables únicos por tarea; matriz de seguimiento
Confundir datos simulados con reales	Medio	Doble reloj (source_t_dat / event_time) e is_synthetic en todo evento

Anexo A. Repositorio y reproducibilidad

El repositorio github.com/estuardoo/STYLUS (privado) se publica sin datos ni secretos. Estructura:

```
config/              JSON de control: parámetros, tablas, columnas, usuarios y permisos
panel/               Panel web local para editar la configuración y desplegar
ops.py               Orquestador de despliegue (Windows, Linux, Cloud Shell)
infra/terraform/     Infraestructura GCP (estado remoto en GCS)
ingestion/batch/     Carga de H&M a Bronze con manifiesto
```

```
ingestion/streaming/    Publicador replay de eventos 2026
pipelines/              Spark (Silver), Dataflow (streaming), recomendador
sql/                    Bronze, Silver, Gold y vistas BI en BigQuery
api/                    FastAPI, web, asistente Gemini y wallet
cost/                   Calculadora y automatización de costos
docs/architecture/      Diagramas regenerables (Graphviz)
docs/entrega1/          Este documento
docs/gestion/           Matriz de auditoría y roles
tests/                  Pruebas unitarias
.github/workflows/      CI: pruebas, costos, terraform validate
data/                   Solo local; excluido de Git
```

Todos los nombres, parámetros, tablas, columnas, usuarios iniciales y permisos se controlan desde config/control_plataforma.json (editable con python ops.py panel); python ops.py config los convierte en variables de Terraform y semillas de la API, y python ops.py verificar compara lo desplegado contra ese archivo.

Comandos principales: python ops.py config · bootstrap · plan · apply · subir-datos · bronze · silver · gold · api · verificar · todo. Guía de despliegue en docs/DEPLOY.md y guía técnica completa en docs/entrega1/Guia_Tecnica_STYLUS.pdf.

Referencias

[1] UTEC. Entregables del Proyecto Final: Taller de Arquitectura Big Data. Sección 1, 2026.

[2] Kaggle. H&M Personalized Fashion Recommendations. https://www.kaggle.com/competitions/h-and-m-personalized-fashion-recommendations

[3] Google Cloud. Cloud Storage pricing. https://cloud.google.com/storage/pricing

[4] Google Cloud. BigQuery pricing. https://cloud.google.com/bigquery/pricing

[5] Google Cloud. Pub/Sub pricing. https://cloud.google.com/pubsub/pricing

[6] Google Cloud. Dataflow pricing. https://cloud.google.com/dataflow/pricing

[7] Google Cloud. Serverless for Apache Spark pricing. https://cloud.google.com/dataproc-serverless/pricing

[8] Google Cloud. Cloud Run pricing. https://cloud.google.com/run/pricing

[9] Google Cloud. Cloud SQL pricing. https://cloud.google.com/sql/pricing

[10] Google Cloud. Vertex AI generative AI pricing. https://cloud.google.com/vertex-ai/generative-ai/pricing

[11] Google Cloud. Pricing Calculator. https://cloud.google.com/products/calculator